Introduction to Logic Synthesis

Using Cadence Genus with Stylus Common UI

For a visual demonstration of the material in this document, please see the tutorials at:
- Simple block - Part 1: https://youtu.be/HoiBsXhU0ro
- Simple block - Part 2: https://youtu.be/igx73KGziPc
- Full SoC Synthesis: https://youtu.be/Xo1Tv0yxPMc

1 Introduction

Digital logic is typically described in a behavioral register-transfer level (RTL) description. This is *synthesizable* code (i.e., it can be turned into Boolean logic), but it is technology independent and is human readable. Writing behavioral RTL code is much more compact, convenient, and user-friendly than writing a netlist of logic gates. The job of the synthesis tool is to translate the RTL code into a technology specific netlist of standard cells and on the way, optimize the implementation for minimal area, power and timing.

1.1 What is needed to perform Synthesis?

- **RTL Code**: This is the code you write, typically in Verilog or VHDL.

Be careful! Verilog and VHDL contain many more options than the limited set of synthesizable logic permits. Follow coding guidelines to ensure that your HDL is synthesizable!

- **Target Library**: This is the standard cell library that contains the set of logic gates to map the design to.
- **IP Libraries**: These are hard macros (such as SRAMs) that have been provided by a third party and are instantiated in the RTL but are not to be synthesized.
- **Design constraints**: These are directives, typically written in synopsys design constraints (SDC) format, that instruct the tool how to optimize the design. The most important of these directives is the clock definition including the target clock frequency.

1.2 Typical Synthesis flow

1. **Read Libraries**: Read the standard cell libraries that will be used to synthesize the design. In some cases, depending on the design, also read in libraries for the macro-cells.
2. **Read Design**: Read the HDL (VHDL/Verilog) files that will be synthesized
3. **Elaborate**: Elaborate a generic netlist which implements the Boolean functionality described in the HDL model and performs Boolean optimization to minimize literals.
4. **Read Constraints**: Read in the SDC file defining the design constraints (target clock period, clock uncertainty, input and output delays, drive strength, output load)
5. **Synthesize to Generics**: Map the elaborated logic onto generic logic gates and perform initial optimization.
6. **Technology Mapping**: Perform technology mapping from generic gates to real standard-cells and optimize the design according to the constraints defined in the SDC file.

7. **Report and Export**: Write out reports about the synthesis results (most importantly area and power) and write out the final results (e.g., gate level netlist).

2 Workspace Setup

The workspace setup is overviewed in the document "**Overview of Project Workspace**", provided separately.

3 Getting Started with Genus

Genus is Cadence's synthesis tool that replaced "**RTL Compiler**". This manual will introduce you to **Genus** using the new ***Stylus Common UI*** commands. This means that anything that was written for **RTL Compiler** or **Genus** "in legacy UI mode" will not work, so pay attention when reading documentation and support answers.

3.1 Starting Genus

To start **Genus** from a **Linux** command line:

```
~/workspace> genus
```

The command prompt will now change to the **Genus** Shell:

```
genus@root:>
```

3.1.1 Log files

Genus will automatically create two log files:

- `genus.log` : the log of all the messages printed to standard output.
- `genus.cmd` : the log of all the command run in the current session.

You can change the name of the log file, using the `-log` option when invoking genus.

Always search for ***warnings*** and ***errors*** in the log file to make sure you didn't miss anything important. A good approach to do this is using the **Linux** `grep` command:

```
grep -ei "warning\|error" genus.log | less
```

It is very useful to use the `.cmd` file to see which commands were run (especially if choosing a GUI menu option) and copying them to a script file.

3.1.2 Tool Command Language (TCL)

The **Genus** Shell interprets **TCL** language. It is highly advised to learn some **TCL**, since you'll be scripting quite a bit.

3.1.3 Getting Help

To get information about a specific command, use the `man` command:

```
man <command name>
```

To find commands that match a certain string, use the help command:

```
help <glob string>
```

Note that help can also be run on an attribute!

3.1.4 Documentation

In addition, you can find the documentation under the `doc` directory of the installation path. The installation path is saved in the `$GENUSHOME` environment variable:

```
%> echo $GENUSHOME
   /apps/cadence/GENUS/21.15/
```

```
%> acroread $GENUSHOME/doc/genus_user/genus_user.pdf &
```

The most important documents are:

- Genus User Guide: `$GENUSHOME/doc/genus_user/genus_user.pdf`
- Genus Command Reference: `$GENUSHOME/doc/genus_comref/genus_comref.pdf`
- Genus Attribute Reference: `$GENUSHOME/doc/genus_attref/genus_attref.pdf`
- ChipWare Reference: `$GENUSHOME/doc/genus_chipwareref/genus_chipwareref.pdf`

To learn about an error/warning/info message:

```
report_messages
```

or

```
man <message_name>
```

To open up Cadence `help` tool, from the Linux shell, type:

```
cdnshelp
```

Finally, you will find the best answers on the Cadence support site: support.cadence.com

4 Genus Virtual Hierarchy

Stylus Common UI (i.e., the *newer* versions of Cadence digital tools) has its own *virtual hierarchy*, which is accessed similar to UNIX paths with `vls`, `vcd` and `vpwd`. This hierarchy has a root directory (`genus:root`) with several primary subdirectories:

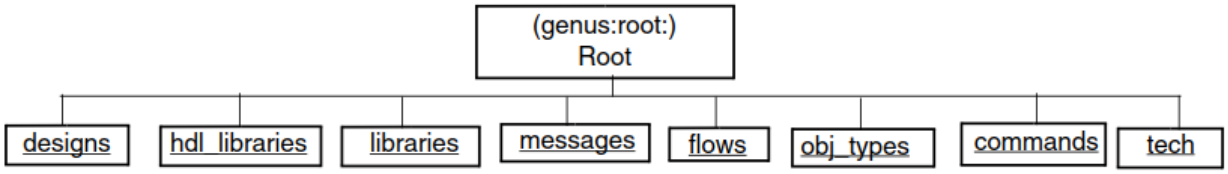


Figure 1: Stylus Common UI Virtual Hierarchy

- `designs` : The `designs` directory contains all the designs read in during a **Genus** session. A design corresponds to a module in Verilog that is **not instantiated**. In other words, it is the top-level Verilog module. The `designs` directory is populated during elaboration and used **after elaboration**.
- `hdl_libraries` : All HDL code and libraries used for HDL code, including **DesignWare** and **ChipWare** IPs.
- `libraries` : All libraries (`.libs`) read in and used for synthesis
- `messages` : Contains all messages displayed during a **Genus** session.
- `flows` : a repository for flows and flow steps.
- `obj_types` : Contains all the object types in the Design Information Hierarchy.
- `commands` : contains the list of all commands available in **Genus**.
- `tech` : Contains information obtained from **LEF**. It has three subdirectories named layers, sites and vias and are populated with information from **LEF**.
- `mmmc_designs_spec` : only relevant if **MMMC** flow is used

4.1 Basic Virtual Hierarchy Commands

4.1.1 The `vls` command

To list the virtual hierarchy, use the `vls` command. So, for example, to list all the pins of a library cell called `INVX8` in the library `MYLIB`:

```
genus:root/> vls /libraries/MYLIB/libcells/INVX8
./ I/     VDD/   VSS/   Z/
```

4.1.2 The vcd command

To change the current working directory in the virtual hierarchy, use the `vcd` command.

```
genus:root/> vcd libraries/
```

4.1.3 Database Manipulation

The biggest change in Cadence tools that was introduced with the *Stylus Common UI* approach is the introduction of the virtual hierarchy and `get_db`/`set_db` commands to all tools (**Genus**, **Innovus**, **Tempus**). It takes a bit of time to get used to these commands, but it is very worth while to learn how to use them in order to become an advanced user.

To see a property in the **Genus** database, use the `get_db` command and to change the value of such a property, use the `set_db` command. These commands will be used throughout this manual, but to get started, here are some useful examples:

- Find all the designs: `get_db designs`
- Find all sequential leaf instances: `get_db insts -if .is_sequential`
- Find all flop leaf instances: `get_db insts -if {.is_flop}`
- Find all hierarchical instances: `get_db hinsts`
- Find all combinational leaf instances: `get_db insts -if .is_comb`
- Find all input ports: `get_db ports -if {.direction == in}`
- Find all input pins on hierarchical instances: `get_db hpins -if {.direction == in}`
- Find pins on a leaf instance: `get_db <inst_name> .pins`
- Get base_cell attribute on a leaf instance: `get_db <inst_name> .base_cell`
- Find all leaf seq instances locally under m1: `get_db design:m1 .local_insts -if .is_seq`

5 Basic Synthesis Flow in Genus

As previously mentioned, logic synthesis basically consists of the following stages:

1. Read Libraries
2. Read Design
3. Elaborate
4. Read Constraints
5. Synthesis (to generics and technology mapping)
6. Create reports
7. Export Design

The following subsections will go over the basic commands and properties to perform all these stages in Genus with Common UI.

5.1 Reading the Libraries

Genus needs libraries (`.lib` files) to describe the Boolean functionality, timing, area, etc. of each of the components it will use to build the mapped netlist (e.g., standard cells, memories, I/Os, etc.). These are defined by using the `read_libs` command and `init_lib_search_path` property.

- `init_lib_search_path` : This property provides Genus with a list of directories to look for `.lib` files in.

```
set_db init_lib_search_path ". <path to standard cell library>"
```

- `library` : This property tells **Genus** which specific libraries to read. You can provide the full path to each file, or just the file name if the path is defined by `init_lib_search_path` .

```
read_libs <list of standard cell library names and corners>
```

You can see which libraries are loaded by querying the `library` property:

```
@genus:root: 72> get_db library
   { .../something.lib }
```

5.2 Define Physical Synthesis Properties

For advanced nodes, it is highly recommended to run physically aware synthesis, which run placement and trial route to get much more accurate parasitic estimation then obsolete wireload models. Physical synthesis needs physical abstract (`.lef`) descriptions of libraries, as well as RC extraction rules (`qrctech` files).

- `read_physical -lef` : This property defines the `.lef` files of the libraries

```
read_physical -lef <list of LEF files>
```

- `qrc_tech_file` : This property defines the RC extraction rules

```
read_qrc <QRC tech file>
```

You can see which `.lef` files are loaded by querying the `init_lef_files` property:

```
@genus:root: 73> get_db init_lef_files
   { .../lef/something.lef }
```

And see what the **QRCTechfile** is by querying the `qrc_tech_file` property:

```
@genus:root: 74> get_db rc_corner:tc_rc_corner .qrc_tech_file
    .../qrcTechFile
```

5.2.1 Floorplan based physical-aware design

For better results, you should define a floorplan (e.g., extracted from an initial **Innovus** run). This enables you to run the `syn_generic` and `syn_map` steps already with physical awareness and provides much better post layout correlation.

Read in a floorplan with the `read_def` command:

```
read_def $design(floorplan_def)
```

5.3 MMMC Flow

An alternative to the `read_libs`, `read_physical` commands, described above, the Multi-Mode Multi-Corner (**MMMC**) flow can be applied, already inside **Genus**. This flow will be described in full detail when discussing physical implementation (Innovus), but for now, if you already have a view definition (`.mmmc`) file, you can just read it in:

```
read_mmmc $design(mmmc_view_file)
```

5.4 Reading the Design

The next step is to read in the RTL files. This is done with the `read_hdl` command and based on several properties:

- `init_hdl_search_path` : This property tells **Genus** where to look for RTL files

```
set_db init_hdl_search_path {. ../sourcefiles}
```

- `hdl_language` : This property tells **Genus** what the language of the RTL files is (unless specifically set, using the `read_hdl -language` option). The options are `v2001` (**Verilog**), `sv` (**System Verilog**) and `vhdl` (**VHDL**):

```
set_db hdl_language v2001
```

- `read_hdl` : This command actually reads the RTL files

```
 read_hdl <Verilog file list>
```

There are several useful options for the `read_hdl` command, such as `-language`, `-f`, `-define`, and `-netlist`. To find out how they work, use the documentation or `man` pages.

5.5 Elaborating the design

Design elaboration is compiling the RTL of the design and transforming it into technology-independent constructs. This is basically the stage that you will find that your code is wrong and cannot be compiled. Elaboration is done with the `elaborate` command.

```
elaborate <top module name>
```

There are many properties that control how elaboration is performed and have to be set prior to elaboration; however, we will not go over them in this manual. To learn about these, refer to the documentation. In addition, you can use the `-parameters` command to override the default parameters in the RTL files.

5.5.1 Post Elaboration Checks

There are many post elaboration checks that can be run to see if your RTL actually described what you wanted it to, but at the very least you should check that you have no **unresolved references**. Unresolved references are instances that are referenced in the RTL, but the synthesizer didn't find an RTL description or library (`.lib` file) that describes them. In other words, *binding* failed.

```
check_design -unresolved
```

5.6 Loading design constraints

After a basic mapping of logic generics to a standard cell library, synthesis optimizes the design. But it needs a set of metrics, constraints and design goals to optimize to. The constraints define these goals. We generally put the design constraints in a separate file, written in the **SDC** format. You can either explicitly write out the constraints, or use the `read_sdc` command to read in an external **SDC** file:

```
read_sdc <constraints file .sdc>
```

5.6.1 Post read_sdc checks

Go over the log file to see if there were errors or warnings, while reading the SDC file. Also, after reading the constraints, use the `check_timing_intent` command to see a summary of the exceptions and timing irregularities that **Genus** sees:

```
check_timing_intent -verbose
```

5.6.2 Define Cost Groups

It is highly recommended to divide your design into different sets of paths, also called "**cost groups**". For example, to define the groups `reg2reg`, `in2reg`, `reg2out`, and `in2out`:

```
define_cost_group -name reg2reg -design $TOPLEVEL
 path_group -from [all_registers] -to [all_registers] -group reg2reg -name reg2reg

define_cost_group -name in2reg -design $TOPLEVEL
 path_group -from [all_inputs] -to [all_registers] -group in2reg -name in2reg

define_cost_group -name reg2out -design $TOPLEVEL
 path_group -from [all_registers] -to [all_outputs] -group reg2out -name reg2out

define_cost_group -name in2out -design $TOPLEVEL
 path_group -from [all_inputs] -to [all_outputs] -group in2out -name in2out
```

5.7 Synthesis

The actual action of synthesis is to *map* the *elaborated* design into **generic cells** and then to *map* them into **standard cells** from the library. Finally, an *optimization* process is run to try and improve the synthesis to meet the *constraints*.

The *effort* that the tool puts in at each stage of synthesis trades off the quality of the result with runtime and computation resources. Of course, on a small design, you will not feel this, but when synthesizing a big design, this will have a significant effect. To set the overall synthesis effort of all stages, use the `syn_global_effort` property.

```
set_db / .syn_global_effort <low|medium|high>
```

Alternatively, the effort of each of the three synthesis stages (generic, mapping, optimization) can be set separately with the `syn_generic_effort`, `syn_map_effort`, `syn_opt_effort` properties.

5.7.1 Synthesizing to Generics

The first step is to synthesize into generic cells (**gcells**) with `syn_generic`. This performs RTL optimization on the design.

```
 set_db syn_generic_effort medium
 syn_generic
```

If you have run the floorplan based physical-aware mode, you can add the `-physical` flag to enable physically-aware generic mapping.

```
syn_generic –physical
```

5.7.2 Technology Mapping

The second step of synthesis is technology mapping with `syn_map`. Technology mapping also performs logic optimization with the goal of providing the smallest possible implementation of the design that satisfies the timing requirements.

```
set_db syn_map_effort medium
syn_map
```

If you have run the floorplan based physical-aware mode, you can add the `-physical` flag to enable physically-aware technology mapping.

```
syn_generic –physical
```

Or have Genus create a square floorplan with an 0.7 utilization target with the `-create_floorplan` flag:

```
syn_generic -create_floorplan –physical
```

5.7.3 Post mapping optimization

After technology mapping, additional optimization is run to try and meet the constraints, using the `syn_opt` command:

```
set_db syn_opt_effort medium
syn_opt -physical
```

Also, you can run physical aware synthesis by adding the `-spatial` option. This runs placement, trial route, parasitic extraction and post placement incremental optimizations:

```
syn_opt -spatial
```

Note that physical synthesis optimization works with the non-floorplan (`lef`-based) mode. In other words, you can add the `-physical` flag if you have just read in `.lef` files and the **QRCTechfile**.

5.8 Post Synthesis Reporting

To understand and analyze the results of the synthesis, there are many reports that can be created and saved as text files.

5.8.1 Timing Report

Use the `report_timing` command to generate reports on the timing of the current design. The default timing report generates the detailed view of the most critical path in the current design.

```
report_timing > timing.rpt
```

The timing report provides the following information:

- Type of cell (flop-flop, or, nor, and so on)
- The cell's fanout and timing characteristics (load, slew, and total cell delay)
- Arrival time for each point on the most critical path

Use the `-lint` option to generate timing reports at different stages of synthesis. This option provides a list of possible timing problems due to over constraining the design or incomplete timing constraints, such as not defining all multicycle or false paths.

```
report_timing -lint
```

Note – in Genus, the `report_timing -lint` command is equivalent to `check_timing_intent`.

5.8.2 Area estimation

The area report gives a summary of the area of each component in the current design. The report gives the number of gates and the area size based on the specified technology library. Levels of hierarchy are indented in the report.

```
report_area
Instance Module  Cell Count  Cell Area  Net Area   Total Area
-------------------------------------------------------------
sm                       40    208.320    43.100      251.420
-------------------------------------------------------------
```

5.8.3 Gates report

To generate a report that shows a profile of all library cells inferred during synthesis:

```
report_gates
        Gate         Instances   Area                          Library
-------------------------------------------------------------------------------------
Some_gate               3      18.720                        wc_library
Some_gate2              3      8.640                         wc_library
```

5.9 Design Export

Following synthesis, **Genus** has a design that is mapped to standard cells from the target library and IPs; however, this is in Cadence's internal database format. In order to do anything with the synthesis results, we have to export them to accepted formats that other tools know how to read.

5.9.1 Saving the design for Genus

The `write_design` command writes out the necessary files and information to restore a Genus session.

```
write_design -base_name <my design>
```

To restore the design in **Genus**, just `source` the `<my design>.genus_setup.tcl` file

```
source mydesign.genus_setup.tcl
```

5.9.2 Exporting for Place and Route

To generate all files needed to be loaded in an **Innovus** session, use the `write_design -innovus` command:

```
write_design -innovus -basename <my design>
```

You can then `source` the `<my design>.invs_setup.tcl` file inside **Innovus** and it will import the design.

5.9.3 Netlist Export

The `write_hdl` command exports the design as a **Verilog** netlist.:

```
write_hdl > my_netlist.v
```

5.9.4 Formal Equivalence Checking

```
write_do_lec -revised netlist_file.v > lec.do
exec lec -LPGXL -nogui -do lec.do
```

5.9.5 Writing out timing information

To write out timing information in the standard delay format (SDF), use the `write_sdf` command:

```
write_sdf > $TOPLEVEL.sdf
```

6 ChipWare IP Components

ChipWare is a collection of components, which implement basic design level IP such as adders, multipliers, pipeline units, and so on. These components are available in **Genus** install package in the form of both design models (implementation) and simulation models. You can instantiate these components in your RTL and use the simulation models for functional verification.

For each **ChipWare** component, you can find:

1. A **Data Sheet**, describing the functionality and usage of the component.
2. A **Synthesis Model** for synthesizing **ChipWare** components to a gatelevel netlist
3. A **Verilog** for running logic simulation, when a **ChipWare** component is instantiated

Genus supports instantiation third-party (non-Cadence) IP components. These include many **Synopsys DesignWare** and **GTECH** components. These are translated into their **ChipWare** equivalents during synthesis.

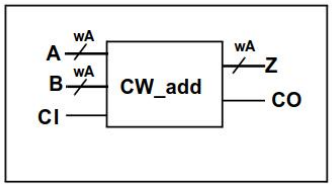
6.1 ChipWare Data Sheets

The datasheets for all **ChipWare** components can be found in the document **"Genus ChipWare IP Components Guide"** under the Genus documentation directory. As an example, let us look at the datasheet of the `CW_add` component.

6.1.1 Component Description

The datasheet tells us what this component is and provides a symbol, showing its inputs and outputs:

Adder



This component performs addition of integer inputs `A` and `B`, and a carry in `CI`, producing output `Z` and carry out `CO`.

Figure 2: Description of the CW_add ChipWare component

6.1.2 Port and Parameter Description

The description is followed by a pair of tables, providing data about the ports (name, input/output, bitwidth, and description) and about the component parameters (name, what the parameter does, the range and the default value). Finally, the Boolean functionality of the component is given:

Port Description

Port Name	Type	Size (bits)	Description
A	Input	wA	Integer input
B	Input	wA	Integer input
CI	Input	1	Carry inp
Z	Output	wA	Integer output
CO	Output	1	Carry out

Parameter Description

Parameter Name	Legal Range	Default	Description
wA	≥1	4	Bit width of A, B, and Z

Functional Description

```
{CO,Z} = A + B + CI
```

Figure 3:Port Description, Parameter Description, and Functionality of the CW_add ChipWare component

6.1.3 Usage of the Component

The final information provided in the datasheet is the usage model for either Verilog or VHDL. This includes an example module that instantiates the component, showing how to declare its ports and parameters. In addition, the path to the simulation model is provided:

Verilog Usage

```
module add (A, B, CI, Z, CO);

    parameter wA = 8;

    input  [wA-1:0] A, B;
    input  CI;
    output [wA-1:0] Z;
    output CO;

    CW_add #(wA)
    U1  (.A   (A),
         .B   (B),
         .CI  (CI),
         .Z   (Z),
         .CO  (CO)
    );
endmodule
```

The Verilog simulation model (`CW_add.v`) is located in the following directory:

```
$CDN_SYNTH_ROOT/lib/chipware/sim/verilog/CW
```

Figure 4: Usage Model of ChipWare Component

6.2 Synthesis of ChipWare Components

ChipWare components can be synthesized in one of two ways:

1. **Instantiation**: Instantiate the **ChipWare** component in your RTL, according to the usage described in the datasheet.

The ChipWare components that are instantiated in the toplevel module are stored in the the `hdl_cw_list` attribute.

2. **Inference**: When a relevant operator/command is used in the RTL, **Genus** will automatically infer the most suitable **ChipWare** component.

The selected ChipWare implementation is both reported in the logfile and stored in the `.selected_impl` database attribute for a module.

6.3 Running Logic Simulation with ChipWare Components

The simulation models for **ChipWare** components are available in the **Genus** installation and should be included with the RTL.

You can identify the available simulation models by checking the `.sim_model` attribute of the ChipWare `hdl_component` component.

For example, to find the simulation model of the CW_add ChipWare components, run:

```
get_db [get_db hdl_component *CW_add] .sim_model
```

6.4 Logic Equivalence

The formal verification tools can use the simulation models of the **ChipWare** components provided with the Genus installation for verification.

For example, to read in the model of the CW_add **ChipWare** component into **Conformal LEC**:

```
vpx read design -verilog2k -merge bbox -golden -lastmod -noelab \
                $CDN_SYNTH_ROOT/lib/chipware/sim/verilog/CW/CW_add.v
```

This document was provided to you courtesy of Prof. Adam Teman of the EnICS Labs Institute in the Alexander Kofkin Faculty of Engineering at Bar-Ilan University. The content was entirely written and compiled by Prof. Teman (without the help of AI tools), based on knowledge and non-proprietary data. You can find all of Prof. Teman's lectures and materials at the EnICS Labs website: https://enicslabs.com/education/

For any comments or enquiries, Prof. Teman can be reached at adam.teman@biu.ac.il